



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Πτυχιακή Εργασία
Σειριακοί και Παράλληλοι Αλγόριθμοι

Καραγιάννης Δημήτριος
1115202200293
sdi2200293@di.uoa.gr

Επιβλέπων Καθηγητής: ΕΔΙΠ Δρ. Λουκά Μαρία

Καλοκαίρι 2026

Περιεχόμενα

Εισαγωγή	3
1 Αλγόριθμοι Γραφημάτων	4
1.1 Ορισμοί και Αναπαράσταση	4
2 Ελάχιστο δέντρο επικάλυψης: Ο αλγόριθμος του Prim	7
2.1 Παράλληλη Υλοποίηση	10
3 Συντομότερες Διαδρομές από Μία Πηγή: Ο αλγόριθμος του Dijkstra	13
3.1 Παράλληλη Υλοποίηση	14
3.1.1 API/Σύστημα μεταβίβασης μηνυμάτων	15
3.1.2 API/Σύστημα κοινής διεύθυνσης	15
3.1.3 Γλώσσα	15
3.1.4 Περιβάλλον συστήματος	15
3.2 Δεδομένα δοκιμών	15
3.3 Αλγόριθμος	16
3.3.1 Υλοποίηση του αλγορίθμου του Dijkstra με MPI	16
3.3.2 Υλοποίηση του αλγορίθμου του Dijkstra με OpenMP	18
4 Αποτελέσματα Υλοποίησης και Ανάλυση	19
5 Συμπεράσματα και Περαιτέρω Έρευνα	25
Βιβλιογραφία	27

Εισαγωγή

Σύντομη εισαγωγή...

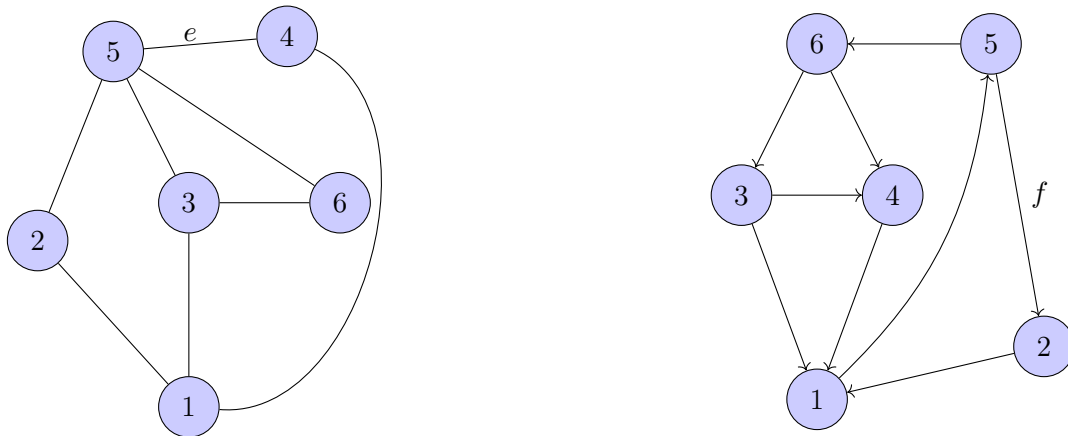
1 Αλγόριθμοι Γραφημάτων

Η θεωρία γραφημάτων διαδραματίζει σημαντικό ρόλο στην επιστήμη των υπολογιστών, επειδή παρέχει έναν εύκολο και συστηματικό τρόπο για τη μοντελοποίηση πολλών προβλημάτων. Πολλά προβλήματα μπορούν να εκφραστούν με όρους γραφημάτων και να επιλυθούν με τη χρήση τυπικών αλγορίθμων γραφημάτων. Το παρόν κεφάλαιο παρουσιάζει παράλληλες υλοποιήσεις κάποιων σημαντικών και θεμελιωδών αλγορίθμων γραφημάτων.

1.1 Ορισμοί και Αναπαράσταση

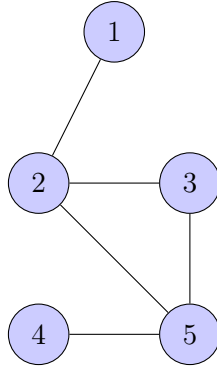
Ένα μη κατευθυνόμενο γράφημα G είναι ένα ζεύγος (V, E) , όπου V είναι ένα πεπερασμένο σύνολο σημείων που ονομάζονται κορυφές και E είναι ένα πεπερασμένο σύνολο ακμών. Μια ακμή $e \in E$ είναι ένα μη διατεταγμένο ζεύγος (u, v) , όπου $(u, v) \in V$. Μια ακμή (u, v) υποδηλώνει ότι οι κορυφές u και v είναι συνδεδεμένες. Ομοίως, ένα κατευθυνόμενο γράφημα G είναι ένα ζεύγος (V, E) , όπου V είναι το σύνολο των κορυφών όπως μόλις ορίσαμε, αλλά μια ακμή $(u, v) \in E$ είναι ένα διατεταγμένο ζεύγος, δηλαδή υποδηλώνει ότι υπάρχει σύνδεση από το u στο v . Το Σχήμα 1 απεικονίζει ένα μη κατευθυνόμενο και ένα κατευθυνόμενο γράφημα. Χρησιμοποιούμε τον όρο γράφημα για να αναφερθούμε τόσο στα κατευθυνόμενα όσο και στα μη κατευθυνόμενα γραφήματα.

Πολλοί ορισμοί είναι κοινοί για κατευθυνόμενα και μη κατευθυνόμενα γραφήματα, αν και ορισμένοι όροι έχουν ελαφρώς διαφορετική σημασία στο καθένα. Αν η (u, v) είναι μια ακμή σε ένα μη κατευθυνόμενο γράφημα, τότε η (u, v) συνδέει (**incident on**) τις κορυφές u και v . Ωστόσο, αν το γράφημα είναι κατευθυνόμενο, τότε η ακμή (u, v) ξεκινάει (**incident from**) από την κορυφή u και καταλήγει (**incident to**) στην κορυφή v . Για παράδειγμα, στο Σχήμα 1(α), η ακμή e συνδέει (**incident on**) τις κορυφές 5 και 4, ενώ στο Σχήμα 1(β), η ακμή f ξεκινάει (**incident from**) από την κορυφή 5 και καταλήγει (**incident to**) στην κορυφή 2. Αν η (u, v) είναι μια ακμή σε έναν μη κατευθυνόμενο γράφημα $G = (V, E)$, οι κορυφές u και v λέγεται ότι είναι γειτονικές (**adjacent**) μεταξύ τους. Αν το γράφημα είναι κατευθυνόμενο, η κορυφή v λέγεται ότι είναι γειτονική της (**adjacent to**) κορυφής u .



Σχήμα 1: (α) Ένα μη κατευθυνόμενο γράφημα και (β) ένα κατευθυνόμενο γράφημα.

Μια διαδρομή (ή μονοπάτι) από μια κορυφή v σε μια κορυφή u είναι μια ακολουθία $< v_0, v_1, v_2, \dots, v_k >$ κορυφών, όπου $v_0 = v$, $v_k = u$, και $(v_i, v_{i+1}) \in E$ για $i = 0, 1, \dots, k-1$.



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Σχήμα 2: Ένα μη κατευθυνόμενο γράφημα και η αναπαράστασή του με λίστα γειτνίασης.

Το μήκος μιας διαδρομής ορίζεται ως το πλήθος των ακμών σε αυτή τη διαδρομή. Αν υπάρχει διαδρομή από το v στο u , τότε το u είναι προσπελάσιμο (**reachable**) από το v . Μια διαδρομή είναι απλή αν όλες οι κορυφές της είναι διακριτές (δεν επαναλαμβάνονται). Μια διαδρομή σχηματίζει κύκλο αν η αρχική και η τελική της κορυφή ταυτίζονται – δηλαδή, αν $v_0 = v_k$. Ένα γράφημα χωρίς κύκλους ονομάζεται ακυκλικό. Ένας κύκλος είναι απλός αν όλες οι ενδιαμέσες κορυφές του είναι διακριτές.

Για παράδειγμα, στο Σχήμα 1(α), η ακολουθία $\langle 3, 6, 5, 4 \rangle$ είναι μια διαδρομή από την κορυφή 3 στην κορυφή 4. Στο Σχήμα 1(β), υπάρχει ένας κατευθυνόμενος απλός κύκλος $\langle 1, 5, 6, 4, 1 \rangle$. Επίσης, στο Σχήμα 1(α), η ακολουθία $\langle 1, 2, 5, 3, 6, 5, 4, 1 \rangle$ είναι ένας μη κατευθυνόμενος κύκλος που δεν είναι απλός, γιατί περιέχει τον βρόχο $\langle 5, 3, 6, 5 \rangle$.

Ένα μη κατευθυνόμενο γράφημα είναι συνεκτικό αν κάθε ζεύγος κορυφών ενώνεται με μια διαδρομή. Λέμε ότι ένα γράφημα $G' = (V', E')$ είναι υπογράφημα του $G = (V, E)$ αν $V' \subseteq V$ και $E' \subseteq E$. Για ένα δεδομένο σύνολο $V' \subseteq V$, το υπόγραφημα του G που προκύπτει από το V' είναι το γράφημα $G' = (V', E')$, όπου $E' = \{(u, v) \in E \mid u, v \in V'\}$. Ένα πλήρες γράφημα είναι ένα γράφημα στον οποίο κάθε ζεύγος κορυφών είναι γειτονικό. Ένα δάσος είναι ένα ακυκλικό γράφημα και ένα δέντρο είναι ένα συνεκτικό ακυκλικό γράφημα. Σημειώστε ότι αν $G = (V, E)$ είναι ένα δέντρο, τότε $|E| = |V| - 1$.

Μερικές φορές τα βάρη συσχετίζονται με κάθε ακμή στο E . Τα βάρη είναι συνήθως πραγματικοί αριθμοί που αντιπροσωπεύουν το κόστος ή το όφελος της διαδρομής κατά μήκος της συνδεδεμένης ακμής. Για παράδειγμα, σε ένα ηλεκτρονικό κύκλωμα, μια αντίσταση μπορεί να αναπαρασταθεί από μια ακμή της οποίας το βάρος είναι η τιμή της αντίστασής της. Ένα γράφημα που έχει βάρη συσχετισμένα με κάθε ακμή ονομάζεται γράφημα με βάρη και συμβολίζεται ως $G = (V, E, w)$, όπου τα V και E είναι όπως ορίσαμε προηγουμένως και η $w : E \rightarrow \mathbb{R}$ είναι μια πραγματική συνάρτηση που ορίζεται στο E . Το βάρος ενός γραφήματος ορίζεται ως το άθροισμα των βαρών των ακμών του. Το βάρος μιας διαδρομής είναι το άθροισμα των βαρών των ακμών της.

Υπάρχουν δύο τυπικές μέθοδοι για την αναπαράσταση ενός γραφήματος σε ένα πρόγραμμα υπολογιστή. Η πρώτη μέθοδος είναι η χρήση πίνακα (μήτρας), και η δεύτερη μέθοδος είναι η χρήση συνδεδεμένης λίστας.

Θεωρήστε ένα γράφημα $G = (V, E)$ με n κορυφές αριθμημένες ως $1, 2, \dots, n$. Ο πίνακας γειτνίασης αυτού του γραφήματος είναι ένας $n \times n$ πίνακας $A = (a_{i,j})$, ο οποίος ορίζεται ως εξής:

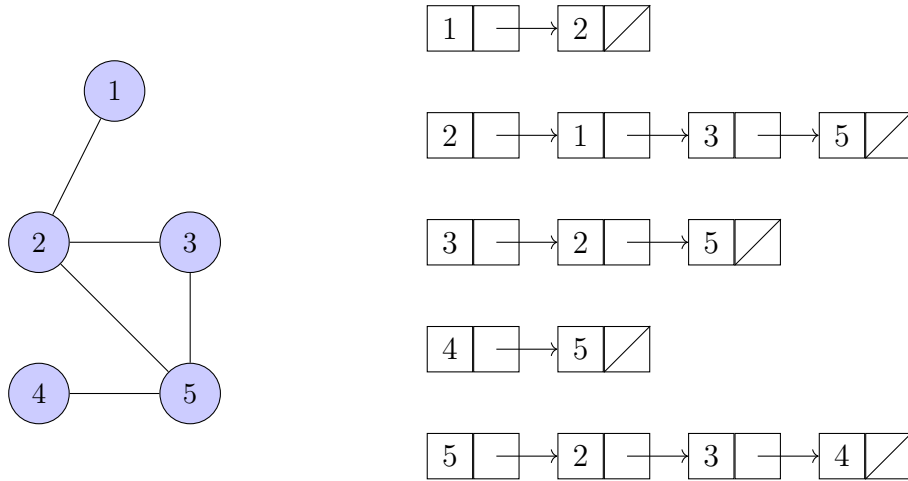
$$a_{i,j} = \begin{cases} 1 & \text{εάν } (v_i, v_j) \in E \\ 0 & \text{διαφορετικά} \end{cases}$$

Το Σχήμα 2 απεικονίζει μια αναπαράσταση με πίνακα γειτνίασης για έναν μη κατευθυνόμενο γράφημα. Σημειώστε ότι ο πίνακας γειτνίασης ενός μη κατευθυνόμενου γραφήματος είναι συμμετρικός. Η αναπαράσταση με πίνακα γειτνίασης μπορεί να τροποποιηθεί για να υποστηρίξει γραφήματα με βάρη. Σε αυτή την περίπτωση, ο $A = (a_{i,j})$ ορίζεται ως εξής:

$$a_{i,j} = \begin{cases} w(v_i, v_j) & \text{εάν } (v_i, v_j) \in E \\ 0 & \text{εάν } i = j \\ \infty & \text{διαφορετικά} \end{cases}$$

Αναφερόμαστε σε αυτόν τον τροποποιημένο πίνακα γειτνίασης ως πίνακα γειτνίασης με βάρη. Ο χώρος που απαιτείται για την αποθήκευση του πίνακα γειτνίασης ενός γραφήματος με n κορυφές είναι $\Theta(n^2)$.

Η αναπαράσταση με λίστα γειτνίασης ενός γραφήματος $G = (V, E)$ αποτελείται από έναν πίνακα $Adj[1..|V|]$ λιστών. Για κάθε $v \in V$, η $Adj[v]$ είναι μια συνδεδεμένη λίστα όλων των κορυφών u για τις οποίες ο G περιέχει μια ακμή $(v, u) \in E$. Με άλλα λόγια, η $Adj[v]$ είναι μια λίστα όλων των κορυφών που είναι γειτονικές προς την v . Το Σχήμα 3 δείχνει ένα παράδειγμα αναπαράστασης με λίστα γειτνίασης. Η αναπαράσταση με λίστα γειτνίασης μπορεί να τροποποιηθεί για να φιλοξενήσει γραφήματα με βάρη, αποθηκεύοντας το βάρος κάθε ακμής $(v, u) \in E$ στη λίστα γειτνίασης της κορυφής v . Ο χώρος που απαιτείται για την αποθήκευση της λίστας γειτνίασης είναι $\Theta(|E|)$.



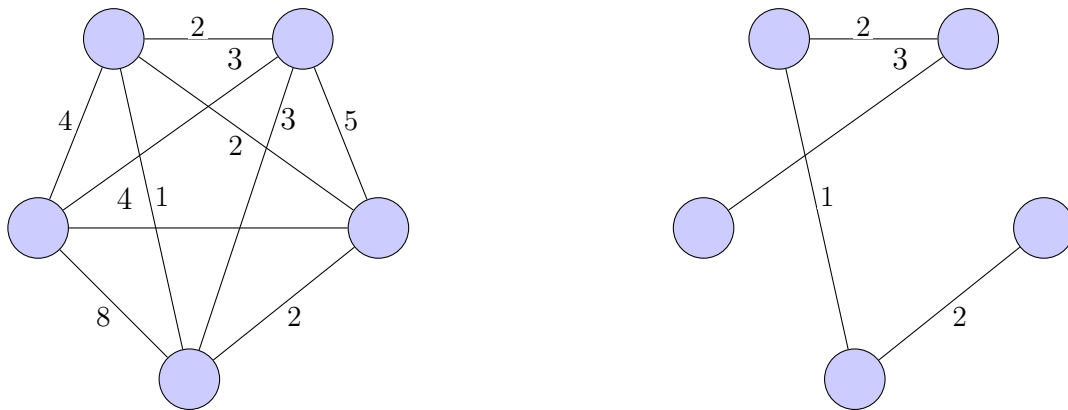
Σχήμα 3: Ένα μη κατευθυνόμενο γράφημα και η αναπαράσταση του με λίστα γειτνίασης.

Η φύση του γραφήματος καθορίζει ποια αναπαράσταση πρέπει να χρησιμοποιηθεί. Ένα γράφημα $G = (V, E)$ είναι αραιό αν το $|E|$ είναι πολύ μικρότερο από $O(|V|^2)$, διαφορετικά είναι πυκνό. Η αναπαράσταση με πίνακα γειτνίασης είναι χρήσιμη για πυκνά γραφήματα, ενώ η αναπαράσταση με λίστα γειτνίασης είναι συχνά πιο αποδοτική για αραιά γραφήματα. Να σημειωθεί ότι ο ακολουθιακός χρόνος εκτέλεσης ενός αλγορίθμου που χρησιμοποιεί πίνακα γειτνίασης και χρειάζεται να διατρέξει όλες τις ακμές του γραφήματος είναι κάτω φραγμένος από $\Omega(|V|^2)$, επειδή πρέπει να προσπελαστεί ολόκληρος ο πίνακας. Ωστόσο, αν χρησιμοποιείται αναπαράσταση με λίστα γειτνίασης, ο χρόνος εκτέλεσης είναι κάτω φραγμένος από

$\Omega(|V| + |E|)$ για τον ίδιο λόγο. Επομένως, αν το γράφημα είναι αραιό (το $|E|$ είναι πολύ μικρότερο από $|V|^2$), η αναπαράσταση με λίστα γειτνίασης είναι καλύτερη από την αναπαράσταση με πίνακα γειτνίασης.

Το υπόλοιπο αυτού του κεφαλαίου παρουσιάζει διάφορους αλγορίθμους γραφημάτων. Τα πρώτα τέσσερα εδάφια παρουσιάζουν αλγορίθμους για πυκνά γραφήματα, και το τελευταίο εδάφιο συζητά αλγορίθμους για αραιά γραφήματα. Υποθέτουμε ότι τα πυκνά γραφήματα αναπαρίστανται με έναν πίνακα γειτνίασης και αραιά γραφήματα με μια λίστα γειτνίασης. Σε όλο αυτό το κεφάλαιο, το n συμβολίζει τον αριθμό των κορυφών στο γράφημα.

2 Ελάχιστο δέντρο επικάλυψης: Ο αλγόριθμος του Prim



Σχήμα 4: Ένα μη κατευθυνόμενο γράφημα και το ελάχιστο δέντρο επικάλυψής του.

Ένα δέντρο επικάλυψης ενός μη κατευθυνόμενου γραφήματος G είναι ένα υπογράφημα του G που είναι ένα δέντρο που περιέχει όλες τις κορυφές του G . Σε ένα γράφημα με βάρη, το βάρος ενός υπογραφήματος είναι το άθροισμα των βαρών των ακμών στο υπογράφημα. Ένα ελάχιστο δέντρο επικάλυψης (MST) για ένα μη κατευθυνόμενο γράφημα με βάρη είναι ένα δέντρο επικάλυψης με ελάχιστο βάρος. Πολλά προβλήματα απαιτούν την εύρεση του MST ενός μη κατευθυνόμενου γραφήματος. Για παράδειγμα, το ελάχιστο μήκος καλωδίου που απαιτείται για τη σύνδεση ενός συνόλου υπολογιστών σε ένα δίκτυο μπορεί να προσδιοριστεί με την εύρεση του MST του μη κατευθυνόμενου γραφήματος που περιέχει όλες τις πιθανές συνδέσεις. Το Σχήμα 4 απεικονίζει ένα MST ενός μη κατευθυνόμενου γραφήματος.

Αν το G δεν είναι συνεκτικό, δεν μπορεί να έχει δέντρο επικάλυψης. Αντ' αυτού, έχει δάσος επικάλυψης. Για λόγους απλότητας στην περιγραφή του αλγορίθμου ελάχιστου δέντρου επικάλυψης, υποθέτουμε ότι το G είναι συνεκτικό. Αν το G δεν είναι συνεκτικό, μπορούμε να βρούμε τις συνεκτικές συνιστώσες του (Ενότητα 7.6) και να εφαρμόσουμε τον αλγόριθμο ελάχιστου δέντρου επικάλυψης σε καθεμία από αυτές. Εναλλακτικά, μπορούμε να τροποποιήσουμε τον αλγόριθμο ελάχιστου δέντρου επικάλυψης έτσι ώστε να παράγει ένα ελάχιστο δάσος επικάλυψης.

```

1: procedure PRIM_MST( $V, E, w, r$ )
2:    $V_T \leftarrow \{r\}$ 
3:    $d[r] \leftarrow 0$ 
4:   for all  $v \in (V - V_T)$  do
5:     if edge  $(r, v)$  exists then
6:        $d[v] \leftarrow w(r, v)$ 
7:     else
8:        $d[v] \leftarrow \infty$ 
9:     end if
10:  end for
11:  while  $V_T \neq V$  do
12:    find vertex  $u$  such that  $d[u] = \min\{d[v] \mid v \in (V - V_T)\}$ 
13:     $V_T \leftarrow V_T \cup \{u\}$ 
14:    for all  $v \in (V - V_T)$  do
15:       $d[v] \leftarrow \min\{d[v], w(u, v)\}$ 
16:    end for
17:  end while
18: end procedure

```

Αλγόριθμος 1: Ο σειριακός αλγόριθμος του Prim για το ελάχιστο δέντρο επικάλυψης.

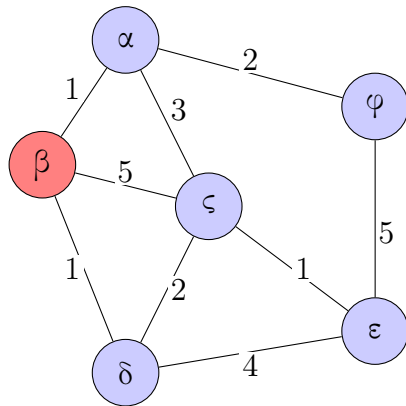
Ο αλγόριθμος του Prim για την εύρεση ενός MST είναι ένας αλγόριθμος απληστίας. Ο αλγόριθμος ξεκινά με την επιλογή μιας τυχαίας αρχικής κορυφής. Στη συνέχεια, αναπτύσσει το ελάχιστο δέντρο επικάλυψης επιλέγοντας μια νέα κορυφή και ακμή που είναι εγγυημένα ότι βρίσκονται σε ένα δέντρο επικάλυψης ελάχιστου κόστους. Ο αλγόριθμος συνεχίζει μέχρι να επιλεγούν όλες οι κορυφές.

Έστω $G = (V, E, w)$ το μη κατευθυνόμενο γράφημα με βάρη για το οποίο πρέπει να βρεθεί το ελάχιστο δέντρο επικάλυψης και έστω $A = (a_{i,j})$ ο πίνακας γειτνίασης με βάρη. Ο αλγόριθμος του Prim απεικονίζεται στον Αλγόριθμο 1. Ο αλγόριθμος χρησιμοποιεί το σύνολο V_T για να διατηρήσει τις κορυφές του ελάχιστου δέντρου επικάλυψης κατά τη διάρκεια της κατασκευής του. Χρησιμοποιεί επίσης έναν πίνακα $d[1..n]$ στον οποίο, για κάθε κορυφή $v \in (V - V_T)$, το $d[v]$ περιέχει το βάρος της ακμής με το ελάχιστο βάρος από οποιαδήποτε κορυφή στο V_T προς την κορυφή v . Αρχικά, το V_T περιέχει μια τυχαία κορυφή r που γίνεται η ρίζα του MST. Επιπλέον, $d[r] = 0$, και για όλα τα v τέτοια ώστε $v \in (V - V_T)$, $d[v] = w(r, v)$ αν υπάρχει τέτοια ακμή, διαφορετικά $d[v] = \infty$. Κατά τη διάρκεια κάθε επανάληψης του αλγορίθμου, μια νέα κορυφή u προστίθεται στο V_T έτσι ώστε $d[u] = \min\{d[v] \mid v \in (V - V_T)\}$. Μετά την προσθήκη αυτής της κορυφής, όλες οι τιμές του $d[v]$ έτσι ώστε $v \in (V - V_T)$ ενημερώνονται, επειδή τώρα μπορεί να υπάρχει μια ακμή με μικρότερο βάρος μεταξύ της κορυφής v και της νέας κορυφής u . Ο αλγόριθμος τερματίζεται όταν $V_T = V$. Το Σχήμα 5 απεικονίζει τον αλγόριθμο. Με την ολοκλήρωση του αλγορίθμου Prim, το κόστος του ελάχιστου δέντρου επικάλυψης είναι $\sum_{v \in V} d[v]$. Ο Αλγόριθμος 1 μπορεί εύκολα να τροποποιηθεί για να αποθηκεύει τις ακμές που ανήκουν στο ελάχιστο δέντρο επικάλυψης.

Στον Αλγόριθμο 1, το σώμα του βρόχου *while* (γραμμές 10-13) εκτελείται $n - 1$ φορές. Τόσο ο υπολογισμός του $\min\{d[v] \mid v \in (V - V_T)\}$ (γραμμή 10), όσο και ο βρόχος *for* (γραμμές 12 και 13) εκτελούνται σε $O(n)$ βήματα. Έτσι, η συνολική πολυπλοκότητα του αλγορίθμου του Prim είναι $\Theta(n^2)$.

Παρακάτω παρουσιάζεται ο αλγόριθμος του Prim. Το MST ξεκινάει από την κορυφή b . Κατά την διάρκεια κάθε επανάληψης, η ακμή με το ελάχιστο κόστος που συνδέει μια κορυφή του V_T με μια κορυφή του $V - V_T$, επιλέγεται και η αντίστοιχη κορυφή προστίθεται στο V_T , (όπως φαίνεται με **έντονη** γραμμή στον πίνακα d). Κατόπιν, ενημερώνονται οι τιμές του πίνακα $d[v]$ των κορυφών του $V - V_T$.

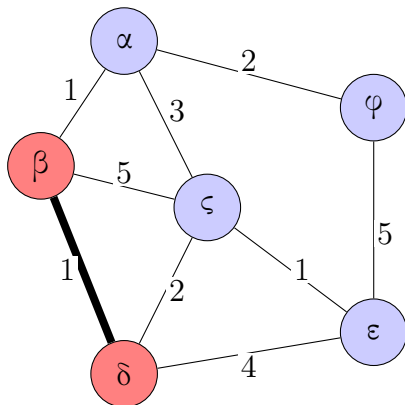
(α) Αρχικό γράφημα



	a	b	c	d	e	f
$d[]$	1	0	5	1	∞	∞

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

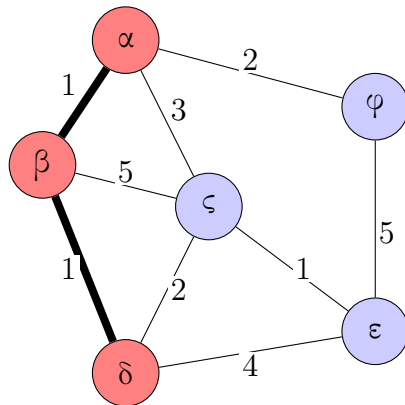
(β) Μετά την επιλογή της πρώτης ακμής



	a	b	c	d	e	f
$d[]$	1	0	2	1	4	∞

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

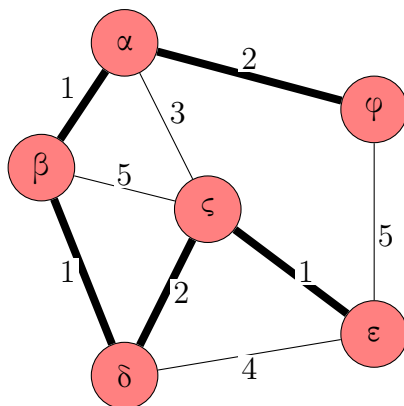
(γ) Μετά την επιλογή της δεύτερης ακμής



	a	b	c	d	e	f
$d[]$	1	0	2	1	4	2

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

(δ) Τελικό ελάχιστο δέντρο επικάλυψης



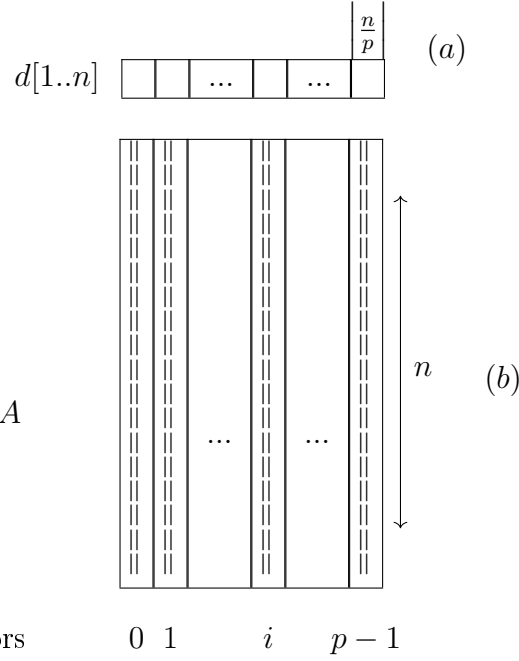
	a	b	c	d	e	f
$d[]$	1	0	2	1	1	2

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

Σχήμα 5: Ο αλγόριθμος του Prim για το ελάχιστο δέντρο επικάλυψης. Το MST έχει ρίζα την κορυφή β. Κατά τη διάρκεια κάθε επανάληψης, επιλέγεται η ακμή ελάχιστου κόστους που συνδέει μια κορυφή στο V_T με μια κορυφή στο $V - V_T$ και η αντίστοιχη κορυφή προστίθεται στο V_T (φαίνεται σκιασμένη στον πίνακα απόστασης d). Στη συνέχεια, οι τιμές $d[v]$ των κορυφών στο $V - V_T$ ενημερώνονται.

2.1 Παράλληλη Υλοποίηση

Ο αλγόριθμος του Prim είναι επαναληπτικός. Κάθε επανάληψη προσθέτει μια νέα κορυφή στο ελάχιστο δέντρο επικάλυψης. Δεδομένου ότι η τιμή του $d[v]$ για μια κορυφή v μπορεί να



Σχήμα 6: Η διαμέριση του πίνακα απόστασης d και ο πίνακας γειτνίασης A μεταξύ p επεξεργαστών.

αλλάζει κάθε φορά που προστίθεται μια νέα κορυφή u στο V_T , είναι δύσκολο να επιλεγούν περισσότερες από μία κορυφές για να συμπεριληφθούν στο ελάχιστο δέντρο επικάλυψης. Για παράδειγμα, στο γράφημα του Σχήματος 5, έπειτα από την επιλογή της κορυφής b , εάν και οι δύο κορυφές d και c επιλεγούν, το MST δεν θα βρεθεί. Αυτό συμβαίνει γιατί, έπειτα από την επιλογή της κορυφής d , η τιμή της $d[c]$ ενημερώνεται από 5 σε 2. Επομένως, δεν είναι εύκολο να εκτελεστούν παράλληλα διαφορετικές επαναλήψεις του βρόχου *while*. Ωστόσο, κάθε επανάληψη μπορεί να εκτελεστεί παράλληλα ως εξής.

Έστω p ο αριθμός των επεξεργαστών και n ο αριθμός των κορυφών στο γράφημα. Το σύνολο V χωρίζεται σε p υποσύνολα χρησιμοποιώντας την *block* διαμέριση σε λωρίδες. Κάθε υποσύνολο έχει $\frac{n}{p}$ συνεχόμενες κορυφές και η εργασία που σχετίζεται με κάθε υποσύνολο ανατίθεται σε διαφορετικό επεξεργαστή. Έστω V_i το υποσύνολο των κορυφών που έχουν ανατεθεί στον επεξεργαστή P_i για $i = 0, 1, \dots, p - 1$. Κάθε επεξεργαστής P_i αποθηκεύει το τμήμα του πίνακα d που αντιστοιχεί στο V_i (δηλαδή, ο επεξεργαστής P_i αποθηκεύει το $d[v]$ έτσι ώστε $v \in V_i$). Το Σχήμα 6(α) απεικονίζει την διαμέριση. Κάθε επεξεργαστής P_i υπολογίζει $d_i[u] = \min\{d_i[v] \mid v \in (V - V_T) \cap V_i\}$ κατά τη διάρκεια κάθε επανάληψης του βρόχου *while*. Το συνολικό ελάχιστο λαμβάνεται τότε για όλα τα $d_i[u]$ χρησιμοποιώντας την ένα-προς-όλους μετάδοση και αποθηκεύεται στον επεξεργαστή P_0 . Ο επεξεργαστής P_0 διατηρεί τώρα τη νέα κορυφή u , η οποία θα εισαχθεί στο V_T . Ο επεξεργαστής P_0 μεταδίδει το u σε όλους τους επεξεργαστές χρησιμοποιώντας την ένα-προς-όλους μετάδοση. Ο επεξεργαστής P_i που είναι υπεύθυνος για την κορυφή u σημειώνει το u ότι ανήκει στο σύνολο V_T . Τέλος, κάθε επεξεργαστής ενημερώνει τις τιμές του $d[v]$ για τις τοπικές κορυφές του.

Όταν μια νέα κορυφή u εισέρχεται στο V_T , οι τιμές του $d[v]$ για το $v \in (V - V_T)$ πρέπει να ενημερωθούν. Ο επεξεργαστής που είναι υπεύθυνος για το v πρέπει να γνωρίζει το βάρος της ακμής (u, v) . Επομένως, κάθε επεξεργαστής P_i πρέπει να αποθηκεύει τις στήλες του πίνακα γειτνίασης με βάρη που αντιστοιχούν στο σύνολο V_i των κορυφών που έχουν εκχωρηθεί σε αυτόν. Ο χώρος για την αποθήκευση του απαιτούμενου τμήματος του πίνακα γειτνίασης

με βάρη σε κάθε επεξεργαστή είναι $\Theta\left(\frac{n^2}{p}\right)$. Το Σχήμα 6(β) απεικονίζει τον διαμερισμό του πίνακα γειτνίασης με βάρη. Ο υπολογισμός που εκτελείται από έναν επεξεργαστή για την ελαχιστοποίηση και την ενημέρωση των τιμών του $d[v]$ κατά τη διάρκεια κάθε επανάληψης είναι $\Theta\left(\frac{n}{p}\right)$. Η επικοινωνία που εκτελείται σε κάθε επανάληψη οφείλεται στη μονού κόμβου συσσώρευση και στη μετάδοση ενός προς όλους.

Υπερκύβος

Για έναν υπερκύβο με p -επεξεργαστές, η μια-προς-όλους μετάδοση μιας λέξης από έναν προς όλους απαιτεί χρόνο $(t_s + t_w) \log p$. Η εύρεση του συνολικού ελάχιστου μιας λέξης σε κάθε επεξεργαστή απαιτεί τον ίδιο χρόνο. Έτσι, το συνολικό κόστος επικοινωνίας κάθε επανάληψης είναι $\Theta(\log p)$. Ο παράλληλος χρόνος εκτέλεσης αυτής της διατύπωσης δίνεται από

$$T_p = \overbrace{\Theta\left(\frac{n^2}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta(n \log p)}^{\text{επικοινωνία}}$$

Δεδομένου ότι ο ακολουθιακός χρόνος εκτέλεσης είναι $W = \Theta(n^2)$, η επιτάχυνση και η αποδοτικότητα είναι οι εξής:

$$S = \frac{\Theta(n^2)}{\Theta\left(\frac{n^2}{p}\right) + \Theta(n \log p)}$$

$$E = \frac{1}{1 + \Theta\left(\frac{p \log p}{n}\right)} \quad (1)$$

Από την Εξίσωση (1) βλέπουμε ότι για μια βέλτιστη ως προς το κόστος παράλληλη υλοποίηση, χρειαζόμαστε χρόνο $\frac{p \log p}{n} = O(1)$. Έτσι, η υλοποίηση σε υπερκύβο του αλγορίθμου του φορειγνλανγυαγενγλισηΠριμ μπορεί να χρησιμοποιήσει μόνο $p = O(n / \log n)$ επεξεργαστές. Επιπλέον, από την Εξίσωση (1), η συνάρτηση ισο-αποδοτικότητας λόγω επικοινωνίας είναι $\Theta(p^2 \log^2 p)$. Δεδομένου ότι το n πρέπει να αυξάνεται τουλάχιστον τόσο γρήγορα όσο το p σε αυτή τη υλοποίηση, η συνάρτηση ισο-αποδοτικότητας λόγω ταυτόχρονης εκτέλεσης είναι $\Theta(p^2)$. Έτσι, η συνολική ισοαποδοτικότητα αυτής της υλοποίησης είναι $\Theta(p^2 \log^2 p)$.

Πλέγμα

Για ένα πλέγμα επεξεργαστών p , η εύρεση ενός ολικού ελάχιστου και η εκτέλεση μιας μετάδοσης ενός προς όλους απαιτούν χρόνο $\Theta(\sqrt{p})$. Επομένως, ο παράλληλος χρόνος εκτέλεσης του αλγορίθμου Prim είναι

$$T_p = \overbrace{\Theta\left(\frac{n^2}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta(n\sqrt{p})}^{\text{επικοινωνία}} \quad (2)$$

Δεδομένου ότι ο ακολουθιακός χρόνος εκτέλεσης είναι $\Omega = \Theta(n^2)$, η επιτάχυνση και η αποδοτικότητα είναι οι εξής:

$$S = \frac{\Theta(n^2)}{\Theta\left(\frac{n^2}{p}\right) + \Theta(n\sqrt{p})}$$

$$E = \frac{1}{1 + \Theta\left(\frac{p^{1.5}}{n}\right)} \quad (3)$$

Στη Εξίσωση (3) βλέπουμε ότι για έναν αλγόριθμο βέλτιστου κόστους, $p^{1.5}/n = O(1)$. Επομένως, μόνο $O(n^{0.66})$ επεξεργαστές μπορούν να χρησιμοποιηθούν αποτελεσματικά. Επίσης από την Εξίσωση (3), η συνάρτηση ισοαποδοτικότητας λόγω επικοινωνίας είναι $\Theta(p^3)$. Αυτή είναι και η συνολική συνάρτηση ισοαποδοτικότητας.

3 Συντομότερες Διαδρομές από Μία Πηγή: Ο αλγόριθμος του Dijkstra

Για ένα γράφημα με βάρη $G = (V, E, w)$, το πρόβλημα των συντομότερων διαδρομών από μία πηγή είναι η εύρεση της συντομότερης διαδρομής από μια κορυφή $v \in V$ προς όλες τις άλλες κορυφές του V . Μια συντομότερη διαδρομή από u προς v είναι μια διαδρομή ελάχιστου βάρους. Ανάλογα με την εφαρμογή, τα βάρη των ακμών μπορεί να αντιπροσωπεύουν χρόνο, κόστος, ποινή, απώλεια ή οποιαδήποτε άλλη ποσότητα που αθροίζεται σωρευτικά κατά μήκος μιας διαδρομής και πρέπει να ελαχιστοποιηθεί. Στην επόμενη ενότητα παρουσιάζουμε τον αλγόριθμο του Dijkstra, ο οποίος λύνει το πρόβλημα των συντομότερων διαδρομών από μία πηγή τόσο σε κατευθυνόμενα όσο και σε μη κατευθυνόμενα γραφήματα με μη αρνητικά βάρη.

Ο αλγόριθμος του Dijkstra, που βρίσκει τις συντομότερες διαδρομές από μία κορυφή s , είναι παρόμοιος με τον αλγόριθμο του Prim για το ελάχιστο δέντρο επικάλυψης. Όπως και ο αλγόριθμος του Prim, βρίσκει σταδιακά τις συντομότερες διαδρομές από το s προς τις υπόλοιπες κορυφές του G . Είναι επίσης άπληστος. Δηλαδή, σε κάθε βήμα επιλέγει μια κορυφή που φαίνεται ότι βρίσκεται πλησιέστερα προς την πηγή. Ο Αλγόριθμος 2 δείχνει τον αλγόριθμο του Dijkstra. Συγκρίνοντας αυτόν τον αλγόριθμο με τον αλγόριθμο του Prim για το ελάχιστο δέντρο επικάλυψης, παρατηρούμε ότι οι δύο είναι σχεδόν ταυτόσημοι. Η βασική διαφορά είναι ότι, για κάθε κορυφή $u \in (V - V_T)$, ο αλγόριθμος του Dijkstra αποθηκεύει $l[u]$, το ελάχιστο κόστος για να φτάσει στην κορυφή u από την κορυφή s χρησιμοποιώντας κορυφές στο V_T . Ο αλγόριθμος του Prim αποθηκεύει $d[u]$, το κόστος της ακμής ελάχιστου κόστους που συνδέει μια κορυφή στο V_T με την u . Ο χρόνος εκτέλεσης του αλγορίθμου του Dijkstra είναι $\Theta(n^2)$.

```

1: procedure DIJKSTRA_SINGLE_SOURCE_SP( $V, E, w, s$ )
2:    $V_T \leftarrow \{s\}$ 
3:    $l[s] \leftarrow 0$ 
4:   for all  $v \in (V - V_T)$  do
5:     if edge  $(s, v)$  exists then
6:        $l[v] \leftarrow w(s, v)$ 
7:     else
8:        $l[v] \leftarrow \infty$ 
9:     end if
10:  end for
11:  while  $V_T \neq V$  do
12:    find vertex  $u$  such that  $l[u] = \min\{l[v] \mid v \in (V - V_T)\}$ 
13:     $V_T \leftarrow V_T \cup \{u\}$ 
14:    for all  $v \in (V - V_T)$  do
15:       $l[v] \leftarrow \min\{l[v], l[u] + w(u, v)\}$ 
16:    end for
17:  end while
18: end procedure

```

Αλγόριθμος 2: Ο σειριακός αλγόριθμος του Dijkstra για τις συντομότερες διαδρομές από μία πηγή.

3.1 Παράλληλη Υλοποίηση

Η παράλληλη υλοποίηση του αλγορίθμου του Dijkstra για τις συντομότερες διαδρομές από μία πηγή είναι πολύ παρόμοια με την παράλληλη υλοποίηση του αλγορίθμου του Prim για τα ελάχιστα δέντρα επικάλυψης (Ενότητα 2). Ο πίνακας γειτνίασης με βάρη διαιρείται χρησιμοποιώντας την *block-striped* απεικόνιση. Κάθε ένας από τους p επεξεργαστές ανατίθεται σε n/p διαδοχικές στήλες του πίνακα γειτνίασης με βάρη και υπολογίζει n/p τιμές του πίνακα l . Κατά τη διάρκεια κάθε επανάληψης, όλοι οι επεξεργαστές εκτελούν υπολογισμούς και επικοινωνία ανάλογη με εκείνη που εκτελείται στην παράλληλη υλοποίηση του αλγορίθμου του Prim. Ο Πίνακας 1 παρουσιάζει τις μετρικές απόδοσης για τους αλγορίθμους του Prim και του Dijkstra σε συστήματα συνδεδεμένα ως δακτύλιος, πλέγμα και υπερκύβος. Σε όλες τις περιπτώσεις, ο παράλληλος χρόνος εκτέλεσης για μια βέλτιστη υλοποίηση ως προς το κόστος είναι επίσης ο ελάχιστος παράλληλος χρόνος εκτέλεσης που μπορεί να επιτευχθεί σε κάθε αρχιτεκτονική σε ασυμπτωτικούς όρους (Πρόβλημα 7.1).

Πίνακας 1: Επιδόσεις των αλγορίθμων Prim και Dijkstra σε διαφορετικές αρχιτεκτονικές.

Αρχιτεκτονική	Μέγιστος αριθμός επεξεργαστών για $E = \Theta(1)$	Αντίστοιχος παράλληλος χρόνος	Συνάρτηση ισοαποδοτικότητας
Δακτύλιος	$\Theta(\sqrt{n})$	$\Theta(n^{1.5})$	$\Theta(p^4)$
Πλέγμα	$\Theta(n^{0.66})$	$\Theta(n^{1.33})$	$\Theta(p^3)$
Υπερκύβος	$\Theta\left(\frac{n}{\log n}\right)$	$\Theta(n \log n)$	$\Theta(p^2 \log^2 p)$

3.1.1 API/Σύστημα μεταβίβασης μηνυμάτων

Το API μεταβίβασης μηνυμάτων πρέπει να είναι διαθέσιμο σε όλα τα συστήματα στα οποία υλοποιείται και θα πρέπει να είναι όσο το δυνατόν πιο απλό και ευθύ, προτιμητέα να υποστηρίζει συλλογικές λειτουργίες. Το MPI καλύπτει αυτές τις απαιτήσεις, επομένως δεν απαιτείται σοβαρή εξέταση εναλλακτικών κατά την επιλογή του. Η υλοποίηση του MPI είναι δωρεάν, εύκολα διαθέσιμη, και παρέχει συνδέσεις (bindings) για τη γλώσσα C που είναι ευρέως γνωστή.

3.1.2 API/Σύστημα κοινής διεύθυνσης

Το επιλεγμένο API κοινής διεύθυνσης πρέπει να επιτρέπει μεγάλο έλεγχο πάνω στις εργασίες που εκτελούνται ταυτόχρονα. Πρέπει επίσης να παρέχεται δωρεάν. Το OpenMP παρέχει ένα επίπεδο πάνω από τις εγγενείς νηματικές (thread) λειτουργίες για τη διευκόλυνση διαφόρων εργασιών σχετικών με νήματα. Χρησιμοποιώντας τις οδηγίες που παρέχει το OpenMP, ο προγραμματιστής δεν χρειάζεται να εκτελέσει την εργασία αρχικοποίησης του αντικειμένου ιδιοτήτων, να ορίσει παραμέτρους για το νήμα και να διαιρέσει τον χώρο επανάληψης. Αυτή η δυνατότητα είναι ιδιαίτερα χρήσιμη όταν το υποκείμενο πρόβλημα έχει στατικό ή τακτικό διάγραμμα εργασιών. Στο πλαίσιο διαφόρων εφαρμογών, έχει αποδειχθεί ότι το κόστος που σχετίζεται με την αυτόματη δημιουργία κώδικα νημάτων από αυτές τις οδηγίες είναι ελάχιστο.

3.1.3 Γλώσσα

Επιλέγεται τη γλώσσα C λόγω της διαθεσιμότητας μιας σχετικά σταθερής υλοποίησης του MPI για μεταβίβαση μηνυμάτων και της βιβλιοθήκης για OpenMP.

3.1.4 Περιβάλλον συστήματος

Η σειριακή υλοποίηση του αλγορίθμου του Dijkstra, η υλοποίηση του αλγορίθμου του Dijkstra με MPI και η υλοποίηση του αλγορίθμου του Dijkstra με OpenMP, και οι τρεις υλοποιήσεις εκτελούνται στο σύστημα του Minnesota Supercomputing Institute (MSI).

3.2 Δεδομένα δοκιμών

Αφού ορίστηκαν οι τεχνολογίες που θα χρησιμοποιηθούν, χρειάζεται ένα πρόγραμμα σε JAVA για τη δημιουργία δοκιμαστικών δεδομένων. Το πρόγραμμα αυτό παράγει έναν δισδιάστατο πίνακα με τυχαίους αριθμούς στο διάστημα 1 έως 15, που αντιπροσωπεύει το γράφο εισόδου για τον αλγόριθμο του Dijkstra. Τα βάρη δημιουργούνται με τη συνάρτηση Random της JAVA. Έστω G ένας δισδιάστατος πίνακας, όπου $G[i][j]$ αντιπροσωπεύει το βάρος από την κορυφή i προς την κορυφή j . Εάν δεν υπάρχει άμεση σύνδεση, το βάρος ορίζεται σε 9999999, αλλιώς είναι ένας τυχαίος αριθμός μεταξύ 1 και 15. Όταν $i = j$, πρόκειται για την ίδια κορυφή, οπότε ορίζουμε $G[i][j] = 0$ σε αυτή την περίπτωση. Τα βάρη παράγονται τυχαία, και η υπολογιζόμενη απόσταση δεν θα είναι πολύ μεγάλη. Αυτό δεν επηρεάζει τους πειραματικούς μας στόχους, εφόσον χρειαζόμαστε μόνο τα αποτελέσματα από διαφορετικά προγράμματα που χρησιμοποιούν τα ίδια δεδομένα. Θα τρέξω το ίδιο σύνολο δεδομένων στη σειριακή υλοποίηση του αλγορίθμου του Dijkstra, στην υλοποίηση του αλγορίθμου του Dijkstra με MPI και στην υλοποίηση του αλγορίθμου του Dijkstra με OpenMP για να

συγκρίνω τους χρόνους εκτέλεσης. Συνολικά χρησιμοποιούνται έξι σύνολα δεδομένων ως πίνακες γειτνίασης. Αυτό σημαίνει για γράφους με 8, 64, 256, 512, 1024 και 2048 κορυφές. Ακολουθεί ένα παράδειγμα για τη μήτρα των 8 κορυφών.

0	2	9999999	3	4	3	9999999	3
2	0	8	8	9	9999999	7	7
9999999	8	0	6	7	9999999	9999999	2
3	8	6	0	7	3	9	7
4	9	7	7	0	9999999	9999999	4
3	9999999	9999999	3	9999999	0	8	3
9999999	7	9999999	9	9999999	8	0	9999999
3	7	2	7	4	3	9999999	0

3.3 Αλγόριθμος

Στην παρακάτω ενότητα δίνονται λεπτομέρειες σχετικά με το πώς υλοποιήθηκε ο αλγόριθμος. Ο πλήρης πηγαίος κώδικας για τις περιγραφόμενες υλοποιήσεις βρίσκεται στο Παράρτημα Α. Εδώ παρέχεται ψευδοκώδικας που περιγράφει τις υλοποιήσεις σε απλοποιημένη μορφή.

3.3.1 Υλοποίηση του αλγορίθμου του Dijkstra με MPI

Η υλοποίηση του αλγορίθμου, σε απλοποιημένη μορφή σύμφωνα με τον ψευδοκώδικα, παρουσιάζεται παρακάτω.

```

1: procedure DIJKSTRA_SINGLE_SOURCE_SP_MPI( $V, E, wgt, lengths, s$ )
2:   initialize local and global data structures
3:   for all  $v \in V$  do
4:     set  $lminpair[0]$ : local min distance
5:     set  $lminpair[1]$ : corresponding vertex
6:   end for
7:   while not all vertices processed do
8:     for vertices in each processor find a vertex  $u$  with the smallest distance from
       the source  $s$ 
9:     perform MPI_Allreduce to obtain the global minimum across processors
10:    get the global minimum vertex  $u$  and mark it as processed
11:    for all  $v \in nlocals$  do ▷  $nlocals$  = number of vertices stored locally
12:       $lengths[v] \leftarrow \min\{lengths[v], udist + wgt[u * nlocal + v]\}$ 
13:    end for
14:  end while
15: end procedure

```

Αλγόριθμος 3: Ο αλγόριθμος του Dijkstra με MPI για τις συντομότερες διαδρομές από μία πηγή.

Ο βασικός υπολογιστικός βρόχος του παράλληλου αλγορίθμου του Dijkstra για τις συντομότερες διαδρομές από μία πηγή εκτελεί τρία βήματα. Πρώτον, κάθε διεργασία εντοπίζει την τοπικά αποθηκευμένη κορυφή στο V_o με τη μικρότερη απόσταση από την πηγή. Έπειτα, οι διεργασίες καθορίζουν την κορυφή με τη μικρότερη απόσταση και την προσθέτουν στο V_c . Τρίτον, κάθε διεργασία ενημερώνει τον πίνακα αποστάσεων ώστε να ληφθεί υπόψη ότι στο V_c έχουν προστεθεί νέες κορυφές.

Το πρώτο βήμα είναι η σάρωση των τοπικών κορυφών του V_o για να βρεθεί η κορυφή v με τη μικρότερη τοπική απόσταση $[v]$. Το αποτέλεσμα του υπολογισμού αποθηκεύεται στον πίνακα $lminpair$. Συγκεκριμένα, ο $lminpair[0]$ αποθηκεύει την απόσταση μεταξύ των κορυφών και ο $lminpair[1]$ αποθηκεύει τις ίδιες τις κορυφές. Ακολουθώντας τα παρακάτω βήματα θα γίνει κατανοητό γιατί θα πρέπει να χρησιμοποιηθεί αυτή η λύση αποθήκευσης. Το επόμενο βήμα είναι ο προσδιορισμός της κορυφής με τη μικρότερη συνολική απόσταση προς την πηγή, το οποίο γίνεται ελαχιστοποιώντας την τιμή που περιέχεται στο $lminpair[0]$.

Ωστόσο, πέρα από την ίδια την ελάχιστη απόσταση, χρειάζεται και ο δείκτης (η ταυτότητα) της κορυφής που επιτυγχάνει αυτή την απόσταση. Γι' αυτό η κατάλληλη λειτουργία μείωσης είναι η MPI_MINLOC , η οποία επιστρέφει την ελάχιστη τιμή και τον δείκτη που αντιστοιχεί στην ελάχιστη αυτή τιμή. Εξαιτίας της MPI_MINLOC χρησιμοποιούμε πίνακα δύο στοιχείων $lminpair$ για να αποθηκεύσουμε την απόσταση και την κορυφή που την επιτυγχάνει. Επιπλέον, όλες οι διεργασίες χρειάζονται το αποτέλεσμα της διαδικασίας ανάκτησης δεδομένων για να εκτελέσουν το τρίτο βήμα, γι' αυτό χρησιμοποιούμε την $MPI_Allreduce$ για να πραγματοποιήσουμε τη μείωση. Το αποτέλεσμα της μείωσης επιστρέφεται στον πίνακα $gminpair$. Το τρίτο και τελευταίο βήμα κάθε επανάληψης εκτελείται σαρώνοντας τις τοπικές κορυφές που ανήκουν στο V_o και ενημερώνοντας, για κάθε τοπική κορυφή v , την ελάχιστη απόσταση μεταξύ αυτής και της κορυφής-πηγής.

Στο πρόγραμμα MPI, γίνεται ανάθεση σε κάθε επεξεργαστή n/p διαδοχικές στήλες του πίνακα W και χρησιμοποιείται η μείωση MPI_MINLOC για να επιλεγεί η κορυφή v που θα συμπεριληφθεί στο V_c σε κάθε επανάληψη. Αξίζει να σημειωθεί ότι ο δείκτης που επιστρέφει η MPI_MINLOC σε συγκρίσεις όπως (a, i) και (a, j) είναι ο μικρότερος δείκτης όταν οι

τιμές είναι ίδιες. Επομένως, μεταξύ των κορυφών που έχουν την ίδια ελάχιστη απόσταση προς την πηγή, προτιμώνται οι κορυφές με τον μικρότερο δείκτη. Αυτό μπορεί να οδηγήσει σε ανισορροπία φόρτου, αφού οι κορυφές που αποθηκεύονται σε διεργασίες με μικρότερους δείκτες τείνουν να περιλαμβάνονται πρώτες στο V_c σε σχέση με αυτές που βρίσκονται σε διεργασίες με μεγαλύτερους δείκτες (ειδικά όταν πολλές κορυφές στο V_o έχουν την ίδια μικρή απόσταση προς την πηγή). Συνεπώς, στις διεργασίες με μεγαλύτερους δείκτες το διαμορφωμένο μέγεθος του V_o θα είναι μεγαλύτερο και ο συνολικός χρόνος εκτέλεσης μπορεί να κυριαρχείται από αυτές.

Μια λύση σε αυτό το πρόβλημα είναι η κυκλική κατανομή των στηλών του W . Αυτή η διαδικασία κατανομής συγκεντρώνει όλες τις p κορυφές, ξεκινώντας από την κορυφή i . Σε αυτό το πλαίσιο, κάθε διεργασία εξακολουθεί να αναθέτει n/p κορυφές αλλά οι δείκτες τους καλύπτουν σχεδόν ολόκληρο το γράφημα. Έτσι, η *MPI_MINLOC* θα ευνοεί συστηματικά κορυφές με μικρότερους δείκτες και δεν θα προκαλείται ανισορροπία φόρτου [2].

3.3.2 Υλοποίηση του αλγορίθμου του Dijkstra με OpenMP

Οι περισσότερες δομές του OpenMP εφαρμόζονται σε ένα δομημένο μπλοκ, δηλαδή σε ένα μπλοκ μιας ή περισσότερων εντολών με ένα σημείο εισόδου στην κορυφή και ένα σημείο εξόδου στο κάτω μέρος. Καθίσταται εφικτός ο εντοπισμός των υπολογιστικά εντατικών βρόχων στον σειριακό αλγόριθμο του Dijkstra και οι επαναλήψεις του βρόχου γίνονται ανεξάρτητες, ενώ στη συνέχεια μπορεί να γίνει η τοποθέτηση των κατάλληλων οδηγιών του OpenMP και να γίνουν δοκιμές.

```

1: procedure DIJKSTRA_SP_OMP( $V, E, w, distances, s$ )
2:   begin
3:     while not all vertices processed do
4:       #pragma omp parallel private
5:       shared()
6:       omp_get_thread_num()
7:       omp_get_num_threads()
8:       Each thread finds the min distance  $u$  unconnected vertex inner
9:       #pragma omp critical // update overall min
10:      #pragma omp barrier
11:      #pragma omp single // mark new vertex as done
12:      for all  $v$  in each thread do
13:         $distances[v] \leftarrow \min\{distances[v], distances[u] + w[u][v]\}$ 
14:      end for
15:    end while
16: end procedure

```

Αλγόριθμος 4: Ο αλγόριθμος του Dijkstra με OpenMP για τις συντομότερες διαδρομές από μία πηγή.

Όπως δείχνει ο ψευδοκώδικας, η υλοποίηση του αλγορίθμου Dijkstra με OpenMP είναι πολύ παρόμοια με τη σειριακή. Σε σύγκριση με την υλοποίηση με MPI, η υλοποίηση με OpenMP απαιτεί λιγότερες γραμμές κώδικα. Η συνάρτηση *omp_set_num_threads* ορίζει τον προεπιλεγμένο αριθμό νημάτων που θα δημιουργηθούν όταν συναντηθεί η επόμενη οδηγία *parallel*. Χρησιμοποιούμε αυτή τη συνάρτηση στη συνάρτηση *main*. Η

omp_get_num_threads επιστρέφει τον αριθμό των νημάτων που συμμετέχουν σε μια ομάδα. Η *omp_get_thread_num* επιστρέφει ένα μοναδικό αναγνωριστικό για κάθε νήμα στην ομάδα. Αυτός ο ακεραίος παίρνει τιμές από 0 έως *omp_get_num_threads()* - 1. Η οδηγία *critical* διασφαλίζει ότι, σε κάθε χρονική στιγμή κατά την εκτέλεση, μόνο ένα νήμα βρίσκεται μέσα σε μια κρίσιμη ενότητα προσδιορισμένη με ένα συγκεκριμένο όνομα. Η OpenMP παρέχει την οδηγία *critical* για την υλοποίηση κρίσιμων περιοχών. Μια κρίσιμη περιοχή επιτρέπει σε διαφορετικά νήματα να εκτελούν διαφορετικό κώδικα ενώ είναι προστατευμένα μεταξύ τους. Το *barrier* είναι μία από τις πιο συχνά χρησιμοποιούμενες πρωτόγονες λειτουργίες συγχρονισμού. Η OpenMP παρέχει την οδηγία *barrier*. Όταν η εκτέλεση συναντήσει αυτή την οδηγία, όλα τα νήματα της ομάδας περιμένουν μέχρι να φτάσουν τα υπόλοιπα και στη συνέχεια συνεχίζουν. Η οδηγία *single* προσδιορίζει ένα δομημένο μπλοκ που εκτελείται από ένα μόνο νήμα. Κατά την είσοδο στο μπλοκ *single*, το πρώτο νήμα εισέρχεται και όλα τα υπόλοιπα νήματα προχωρούν μέχρι το τέλος του μπλοκ [2].

4 Αποτελέσματα Υλοποίησης και Ανάλυση

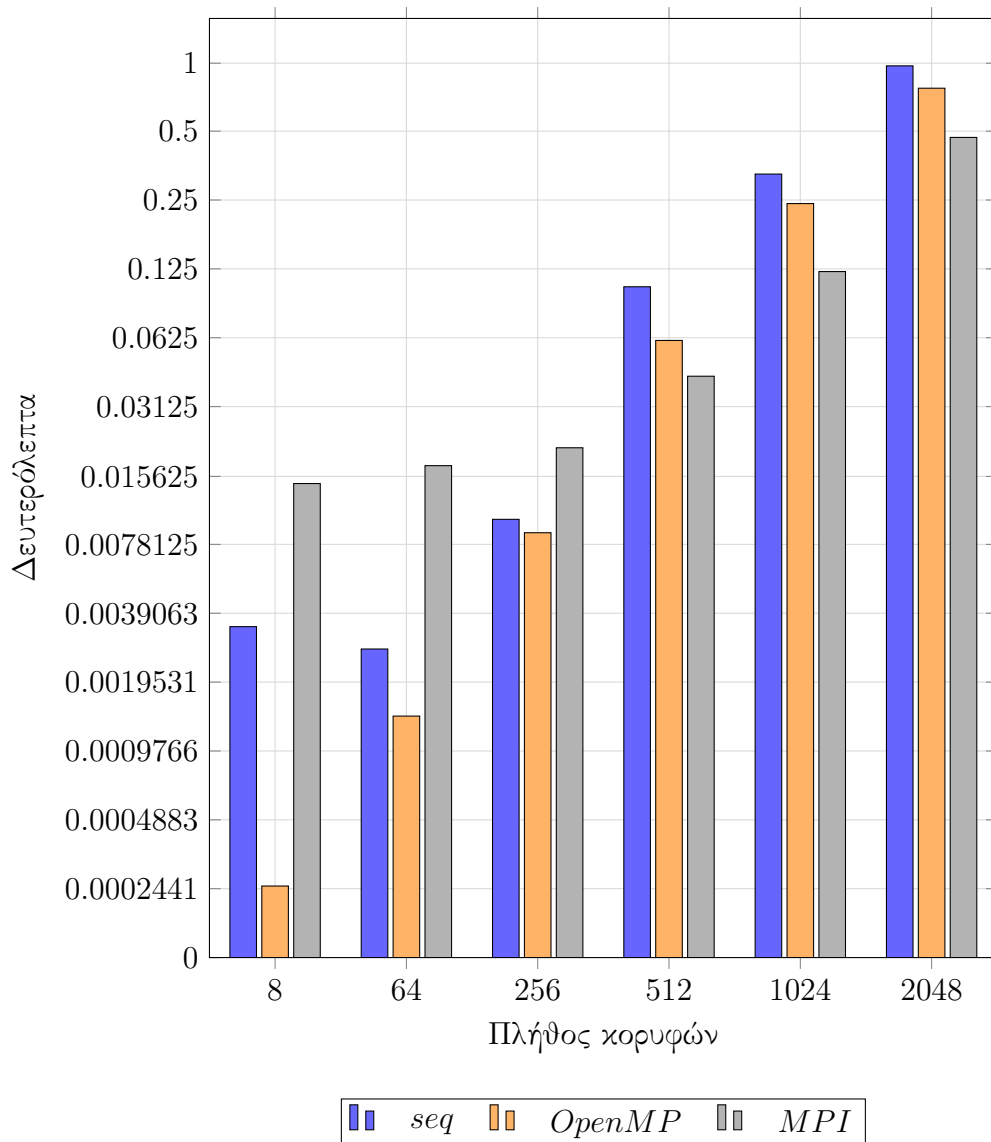
Ο Πίνακας 2 περιέχει όλα τα βασικά αποτελέσματα από την σειριακή εκτέλεση του αλγορίθμου του Dijkstra, τα MPI και OpenMP προγράμματα. Ο κώδικας εκτελέστηκε στο ίδιο περιβάλλον συστήματος και τα δεδομένα εισόδου δημιουργήθηκαν με τη συνάρτηση *Random*. Στην παρούσα εργασία χρησιμοποιούνται συνολικά έξι σύνολα δεδομένων για την είσοδο του πίνακα γειτνίασης. Συγκεκριμένα, για το γράφο χρησιμοποιούνται 8 κορυφές, 64 κορυφές, 256 κορυφές, 512 κορυφές, 1024 κορυφές και 2048 κορυφές. Μετά την εκτέλεση του κώδικα, λαμβάνονται τα αποτελέσματα ως διάρκεια σε δευτερόλεπτα. Για τους παράλληλους υπολογισμούς με OpenMP και MPI, χρησιμοποιήθηκαν 2, 4, 8, 16, 32, 64, 128, 256 και 512 επεξεργαστές (ο αριθμός των κορυφών πρέπει να είναι μεγαλύτερος από τον αριθμό των επεξεργαστών) για την εκτέλεση του κώδικα.

Πίνακας 2: Χρόνος εκτέλεσης σε δευτερόλεπτα και των τριών υλοποιήσεων

	8 κορυφές	64 κορυφές	256 κορυφές	512 κορυφές	1024 κορυφές	2048 κορυφές
<i>Seq</i>	0.0035	0.0029	0.0099	0.1162	0.3568	0.9965
<i>OpenMP2</i>	0.0003	0.0015	0.0093	0.0651	0.2493	0.8583
<i>OpenMP4</i>	0.0003	0.0016	0.0109	0.0635	0.2488	0.7933
<i>OpenMP8</i>	0.0007	0.8831	0.0140	0.0854	0.2807	0.9432
<i>OpenMP16</i>		0.0574	0.0233	0.2736	0.6794	1.7490
<i>OpenMP32</i>		0.0640	0.1186	0.5117	1.3014	2.8708
<i>OpenMP64</i>		0.0864	0.2030	0.7614	1.7558	3.5983
<i>OpenMP128</i>			0.3302	1.3960	2.8929	5.9080
<i>OpenMP256</i>			0.9902	2.7855	5.3655	10.5335
<i>OpenMP512</i>				6.1405	10.6755	20.9920
<i>MPI2</i>	0.0246	0.0178	0.0287	0.1137	0.1316	0.5040
<i>MPI4</i>	0.0353	0.0210	0.0218	0.0413	0.1281	0.4723
<i>MPI8</i>	0.0144	0.0164	0.0264	0.0440	0.1324	1.3663
<i>MPI16</i>		0.0264	0.0453	0.1010	0.2055	3.3525
<i>MPI32</i>		10.8733	1.2415	56.9774	3.9622	21.4067
<i>MPI64</i>		27.2983	3.1004	67.9168	7.1446	41.7286
<i>MPI128</i>			7.3566	19.8329	17.9661	119.9792
<i>MPI256</i>			18.8646	32.8870	44.8353	343.0892
<i>MPI512</i>				488.7678	1733.9085	2399.9986

Πίνακας 3: Ο καλύτερος χρόνος εκτέλεσης σε δευτερόλεπτα και των τριών υλοποιήσεων (οι αριθμοί στις παρενθέσεις δείχνουν το πλήθος των νημάτων/επεξεργαστών

	8 κορυφές	64 κορυφές	256 κορυφές	512 κορυφές	1024 κορυφές	2048 κορυφές
<i>Seq</i>	0.0035	0.0029	0.0099	0.1162	0.3568	0.9965
<i>OpenMP</i>	0.0003(2)	0.0015(2)	0.0093(2)	0.0635(4)	0.2488(4)	0.7933(4)
<i>MPI</i>	0.0144(8)	0.0164(8)	0.0218(4)	0.0413(4)	0.1281(4)	0.4723(4)



Σχήμα 7: Ο καλύτερος χρόνος εκτέλεσης για κάθε υλοποίηση σε διαφορετικά σύνολα δεδομένων

Το Σχήμα 7 δείχνει τον καλύτερο χρόνο εκτέλεσης για τις υλοποιήσεις σειριακά, με OpenMP και με MPI για διαφορετικό αριθμό κορυφών. Γίνεται αντιληπτό ότι η απόδοση είναι καλύτερη όταν η εκτέλεση του αλγορίθμου γίνεται με OpenMP ή MPI. Για μικρό αριθμό κορυφών, μπορεί να δαπανάται περισσότερος χρόνος σε παραλληλοποίηση και συγχρονισμό απ' ό,τι στην εκτέλεση του σειριακού κώδικα. Επομένως, όταν ο αριθμός κορυφών είναι μικρότερος των 512, δεν είναι προφανές ότι η παραλληλοποίηση υπερέχει της σειριακής ε-

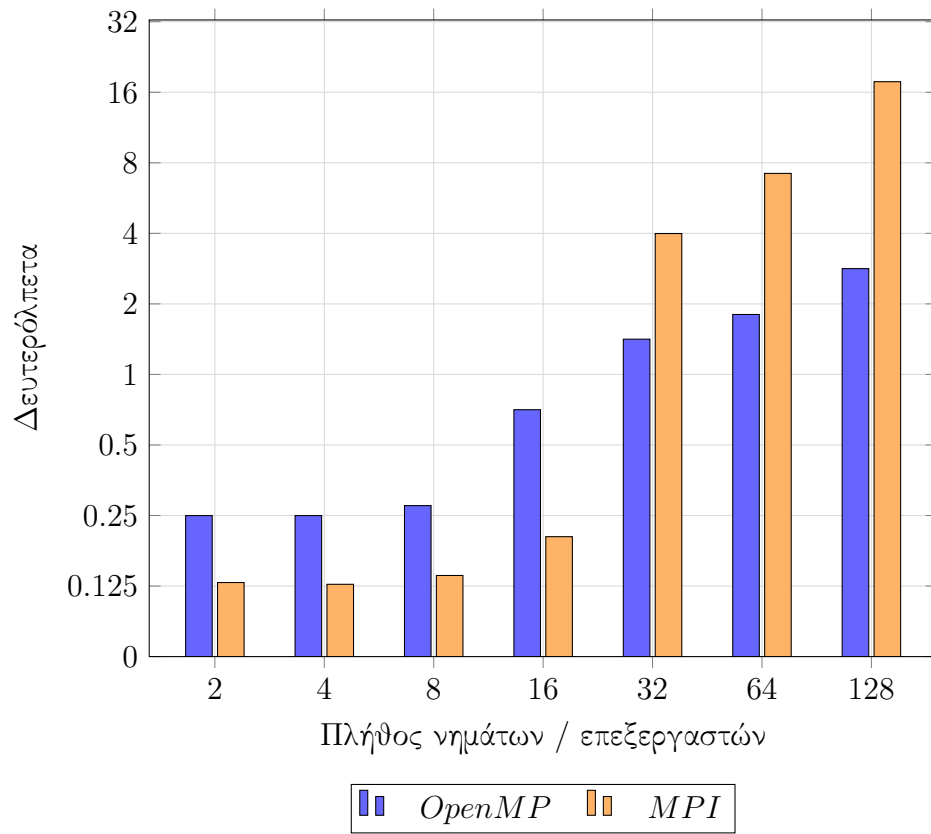
κτέλεσης. Προβλέπεται ότι ο χρόνος εκτέλεσης με MPI και OpenMP θα είναι σημαντικά καλύτερος σε σχέση με τον σειριακό για μεγαλύτερο αριθμό κορυφών.

Ένα ακόμη αποτέλεσμα είναι ότι ο καλύτερος χρόνος εκτέλεσης για το MPI είναι ελαφρώς καλύτερος από τον OpenMP καθώς αυξάνεται ο αριθμός των κορυφών. Σε ένα σύστημα με κοινό χώρο διευθύνσεων, κάθε φορά που ένας επεξεργαστής χρειάζεται να διαβάσει δεδομένα που έχει γράψει άλλος επεξεργαστής, η προσωρινή μνήμη του πρέπει να ενημερωθεί. Όταν πολλοί επεξεργαστές διαβάζουν και γράφουν δεδομένα στην ίδια γραμμή της προσωρινής μνήμης, η προσωρινή μνήμη πρέπει να ενημερώνεται συνεχώς. Αυτό σημαίνει ότι η προσωρινή μνήμη δεν είναι αποτελεσματική καθώς πρέπει να ενημερώνεται διαρκώς. Αυτό μπορεί να έχει μεγάλο αντίκτυπο στην απόδοση αλγορίθμων σε συστήματα με κοινό χώρο διευθύνσεων. Αντίθετα, στα κατανεμημένα συστήματα που χρησιμοποιούν ανταλλαγή μηνυμάτων, κάθε επεξεργαστής διαθέτει ξεχωριστή προσωρινή μνήμη η οποία δεν αδρανοποιείται ή ενημερώνεται άμεσα από άλλους επεξεργαστές. Επομένως, η συνοχή της προσωρινής μνήμης δεν αποτελεί τόσο μεγάλο πρόβλημα στα συστήματα ανταλλαγής μηνυμάτων.

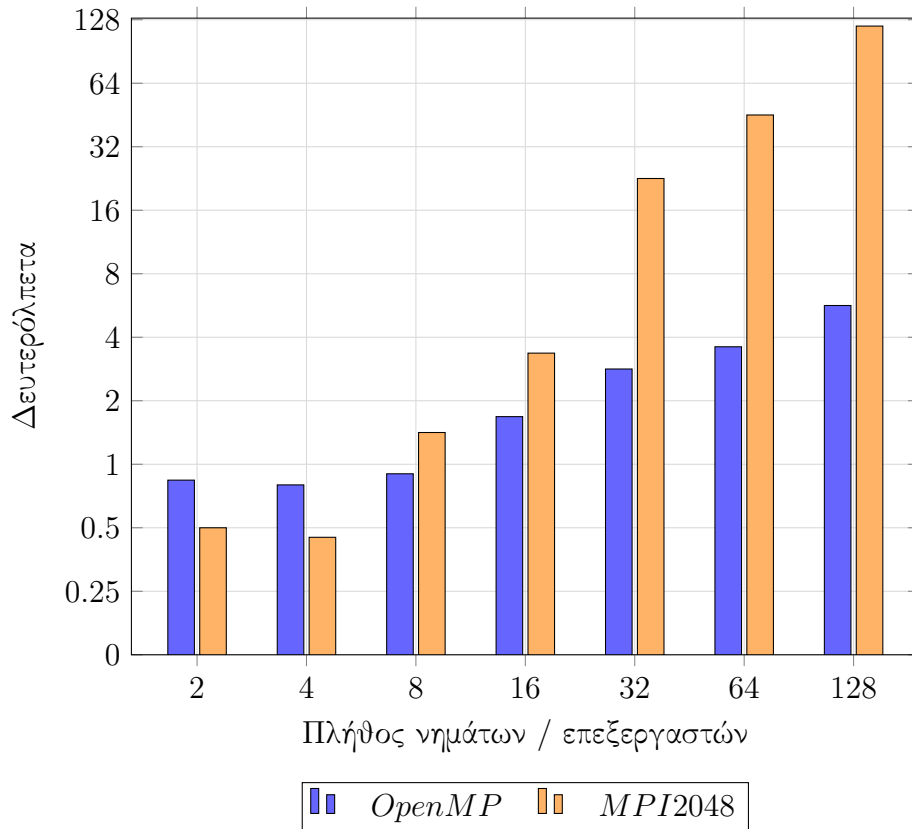
Ο παρακάτω πίνακας δείχνει τον χρόνο υπολογισμού για OpenMP και MPI όταν ο αριθμός κορυφών είναι 1024 και 2048.

Πίνακας 4: Τα αποτελέσματα για το OpenMP και το MPI για διαφορετικά νήματα/επεξεργαστές

πλήθος νημάτων / επεξεργασ- τών	<i>OpenMP</i> 1024 κορυφές	<i>MPI</i> 1024 κορυφές	<i>OpenMP</i> 2048 κορυφές	<i>MPI</i> 2048 κορυφές
2	0.2493	0.1316	0.8583	0.5040
4	0.2488	0.1281	0.7933	0.4723
8	0.2807	0.1324	0.9432	1.3663
16	0.6794	0.2055	1.7490	3.3525
32	1.3014	3.9622	2.8708	21.4067
64	1.7558	7.1446	3.5983	41.7286
128	2.8929	17.9661	5.9080	119.9792



Σχήμα 8: Διαφορετικά νήματα/επεξεργαστές για την υλοποίηση *OpenMP* και *MPI* σε γράφημα με 1024 κορυφές



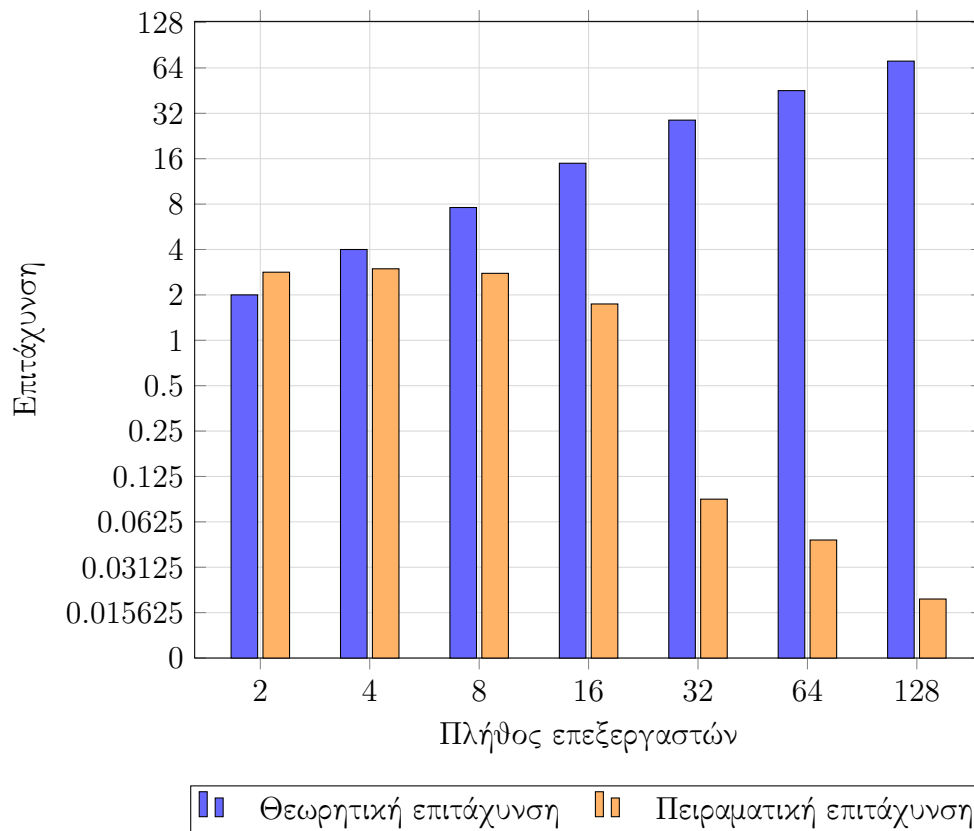
Σχήμα 9: Διαφορετικά νήματα/επεξεργαστές για την υλοποίηση *OpenMP* και *MPI* σε γράφημα με 2048 κορυφές

Το Σχήμα 8 και το Σχήμα 9 παρουσιάζουν την απόδοση του αλγορίθμου του Dijkstra σε διαφορετικούς σχηματισμούς επεξεργαστών/νημάτων. Σύμφωνα με τον Πίνακα 2, είναι γνωστό ότι οι υλοποιήσεις με *OpenMP* και *MPI* αποδίδουν καλύτερα από τις σειριακές υλοποιήσεις για τον αλγόριθμο του Dijkstra. Ωστόσο, από τα Σχήματα 8 και 9 γίνεται αντιληπτό ότι ο μικρότερος αριθμός επεξεργαστών/νημάτων δίνει το ταχύτερο αποτέλεσμα. Για παράδειγμα, αν χρησιμοποιηθούν 1024 κορυφές, ο βέλτιστος αριθμός επεξεργαστών για την υλοποίηση *MPI* είναι 2, 4, 8. Για την *OpenMP* ο αριθμός είναι ο ίδιος. Όταν ο αριθμός των κορυφών είναι 2048, καταλήγουμε σε παρόμοιο συμπέρασμα. Αυτό πιθανώς οφείλεται στο ότι κάθε πρόσθετη διεργασία/νήμα στον κώδικα προκαλεί επιπλέον κόστη επικοινωνίας κατά την ενημέρωσή τους. Καθώς ο αριθμός των επεξεργαστών/νημάτων αυξάνεται εμφανώς, αυτά τα κόστη επικοινωνίας αποκτούν σημαντική επίπτωση. Ειδικότερα για την *MPI* υλοποίηση του αλγορίθμου του Dijkstra, η απόδοση είναι πολύ κακή σε σύγκριση με την αντίστοιχη του *OpenMP*, και η αύξηση του αριθμού των επεξεργαστών εντείνει την επιβράδυνση. Η παράλληλη απόδοση πιθανόν είναι πολύ κακή, διότι κυριαρχείται από τον χρόνο επικοινωνίας, δηλαδή τον χρόνο που απαιτείται για την εκτέλεση του *MPI_Allreduce* σε κάθε επανάληψη.

Σύμφωνα με την παράλληλη υλοποίηση του αλγορίθμου του Dijkstra, είναι δυνατόν να υπολογιστεί η επιτάχυνση σε κάθε συνθήκη. Ο ακόλουθος πίνακας συγκρίνει την θεωρητική επιτάχυνση με την πειραματική επιτάχυνση στην *MPI* υλοποίηση για είσοδο σε γράφο με 1024 κορυφές.

Πίνακας 5: Η σύγκριση της θεωρητικής επιτάχυνσης με την πειραματική επιτάχυνση για την υλοποίηση MPI για γράφημα 1024 κορυφών και διαφορετικό αριθμό επεξεργαστών

Πλήθος επεξεργαστών	Θεωρητική επιτάχυνση για 1024 κορυφές	Πειραματική επιτάχυνση για 1024 κορυφές
2	1.9961	2.7112
4	3.9690	2.7853
8	7.8168	2.6947
16	15.0588	1.7363
32	27.6757	0.0901
64	46.5455	0.0499
128	68.2667	0.0199



Σχήμα 10: Η σύγκριση της θεωρητικής επιτάχυνσης με την πειραματική επιτάχυνση για την υλοποίηση MPI για γράφημα 1024 κορυφών σε διαφορετικούς επεξεργαστές

Η επιτάχυνση είναι μέτρο που αποτυπώνει το σχετικό όφελος της επίλυσης ενός προβλήματος σε παραλληλία. Ορίζεται ως ο λόγος του χρόνου που απαιτείται για την επίλυση ενός προβλήματος με τον καλύτερο σειριακό αλγόριθμο προς τον χρόνο που απαιτείται για την επίλυση του ίδιου προβλήματος σε έναν παράλληλο υπολογιστή με p όμοια στοιχεία επεξεργασίας. Αν η επιτάχυνση μπορεί να διατηρήσει γραμμική αύξηση με τους επεξεργαστές, πολλαπλές μηχανές μπορούν να μειώσουν σημαντικά τον απαιτούμενο χρόνο. Ωστόσο, αυτή η επιτάχυνση είναι πολύ δύσκολο να επιτευχθεί, γιατί με την αύξηση του πλήθους των μηχανών προκύπτουν προβλήματα απωλειών επικοινωνίας, καθώς και προβλήματα από τους

ίδιους τους κόμβους (Διαφορά ρυθμού χρονισμού - the skew of the slaves). Για παράδειγμα, ο συνολικός χρόνος εκτέλεσης του αλγορίθμου καθορίζεται συνήθως από την πιο αργή μηχανή. Αν ο χρόνος που απαιτείται από κάθε υπολογιστή διαφέρει, τότε υπάρχει το πρόβλημα της διαφοράς ρυθμού χρονισμού.

Υπάρχουν πολλές πιθανές αιτίες για τις οποίες το υψηλό επίπεδο παραλληλίας στην υλοποίηση του Dijkstra δεν επέφερε την αναμενόμενη επιτάχυνση. Μία αιτία μπορεί να είναι ότι ο χρησιμοποιούμενος κώδικας είναι αναποτελεσματικός.

Στο πείραμα, η μείωση της επιτάχυνσης μπορεί επίσης να οφείλεται στο ότι η λανθάνουσα καθυστέρηση επικοινωνίας υπερσχύει του οφέλους από τη χρήση περισσότερων επεξεργαστών. Πρέπει να ληφθούν υπόψη όλες οι πληροφορίες που απαιτούνται για την αξιολόγηση της απόδοσης ενός παράλληλου αλγορίθμου σε μια συγκεκριμένη αρχιτεκτονική με συγκεκριμένες τεχνολογικά εξαρτώμενες σταθερές, όπως η ταχύτητα της Κεντρικής Μονάδας Επεξεργασίας (ΚΜΕ - CPU) και η ταχύτητα επικοινωνίας.

5 Συμπεράσματα και Περαιτέρω Έρευνα

Σε αυτή την εργασία εισήχθη, σχεδιάστηκε και υλοποιήθηκε ο παράλληλος αλγόριθμος του Dijkstra. Τα ευρήματα οδηγούν στα ακόλουθα συμπεράσματα:

- Η απόδοση του αλγορίθμου του Dijkstra είναι καλύτερη όταν χρησιμοποιούνται οι υλοποιήσεις με OpenMP και MPI σε σύγκριση με τη σειριακή υλοποίηση, ιδιαίτερα για μεγάλα σύνολα εισόδου.
- Στην παραλληλοποίηση του αλγορίθμου του Dijkstra, ο καλύτερος χρόνος εκτέλεσης για την MPI είναι ελαφρώς καλύτερος από αυτόν της OpenMP όσο αυξάνεται ο αριθμός των κορυφών.
- Και για τις δύο υλοποιήσεις OpenMP και MPI, ο μικρότερος αριθμός επεξεργαστών/νημάτων δίνει το ταχύτερο αποτέλεσμα.
- Συγκρίνοντας τη θεωρητική επιτάχυνση με την πειραματική επιτάχυνση στην υλοποίηση MPI με είσοδο 1024 κορυφών, η πειραματική επιτάχυνση δεν παρουσιάζει γραμμική αύξηση με την αύξηση των επεξεργαστών. Σε σύγκριση με τη σειριακή εκτέλεση, η παράλληλη εκτέλεση του αλγορίθμου του Dijkstra δεν εμφανίζει ικανοποιητική απόδοση όσον αφορά την επιτάχυνση.

Τα ακόλουθα θα ήταν χρήσιμα θέματα για περαιτέρω έρευνα:

- Βελτιστοποίηση των υλοποιήσεων του αλγορίθμου. Πλήρως βελτιστοποιημένες υλοποιήσεις, και ιδιαίτερα η χρήση ουράς προτεραιότητας αντί για πίνακα, αποτελούν πεδίο που επιτρέπει σημαντική περαιτέρω εργασία. Μια ουρά προτεραιότητας είναι τέτοια όπου κάθε στοιχείο έχει επιπλέον μια «προτεραιότητα». Σε μια ελάχιστη ουρά προτεραιότητας, το ελάχιστο στοιχείο έχει την υψηλότερη προτεραιότητα και εξυπηρετείται πριν από ένα στοιχείο με χαμηλότερη προτεραιότητα. Μια ελάχιστη ουρά προτεραιότητας παρέχει τρεις βασικές λειτουργίες: *add_with_priority()*, *decrease_priority()* και *extract_min()*. Έστω $|V|$ ο αριθμός κορυφών και $|E|$ ο αριθμός ακμών ενός γράφηματος. Αν ο αλγόριθμος του Dijkstra χρησιμοποιεί πίνακα για να σαρώσει άμεσα όλες τις κορυφές, το κόστος είναι $O(|V|^2)$. Για αραιά γράφηματα, εάν ο αριθμός των ακμών είναι μικρότερος από τον αριθμό των κορυφών, η υλοποίηση του γραφήματος εισόδου

μπορεί να γίνει με λίστα γειτνίασης αντί για πίνακα γειτνίασης και να χρησιμοποιηθεί δυαδικός σωρός ή σωρός Fibonacci ως ουρά προτεραιότητας για βελτιστοποίηση.

Η διαδικασία είναι η εξής:

1. Προσθήκη της κορυφής πηγής στον σωρό και αναπροσαρμογή του σωρού.
2. Επιλογή του κορυφαίου στοιχείου u και διαγραφή του από τον σωρό.
3. Επεξεργασία των κορυφών που είναι γειτονικές του u : αν μια κορυφή βρίσκεται στην ουρά, ενημέρωση της απόστασης και αναπροσαρμογή της θέσης του στοιχείου στον σωρό. Αν η κορυφή δεν βρίσκεται στον σωρό, προσθήκη της στον σωρό και αναπροσαρμογή του σωρού.
4. Αν το προκύπτον u είναι το τελικό σημείο, τερματίζει ο αλγόριθμος. Αλλιώς επανάληψη των βημάτων 2 και 3.

Η πολυπλοκότητα με χρήση δυαδικού σωρού απαιτεί $O((|E| + |V|) \log |V|)$. Ο σωρός Fibonacci βελτιώνει αυτό σε $O(|E| + |V| \log |V|)$.

- Εκτέλεση πειραματικών δοκιμών σε διαφορετικά περιβάλλοντα συστήματος.

Βιβλιογραφία

- [1] Vipin Kumar et al. *Introduction to Parallel Computing: Design and Analysis of Algorithms*. Addison-Wesley, 2003.
- [2] Thomas H. Cormen et al. *Introduction to Algorithms*. MIT Press, 2009.
- [3] Mengqing He. «Parallelizing Dijkstra's Algorithm». Master's thesis. St. Cloud State University, Dec. 2020. URL: https://repository.stcloudstate.edu/csit_etds/35.